Object Oriented Programming

Chapter 5:

Inheritance

Objectives

At the end of this Lecture, students will become familiar with:

- Base Class Vs Derived Class;
- "protected" Access Specifier;
- Constructor and Destructor in The Inheritance;
- Using "base“ Keyword;
- Overriding the ToString Method;
- Multiple Inheritance Vs Single Inheritance.

Outline

- Inheritance Concept
- "protected" Access Specifier
- Virtual Methods And Overriding
- Multiple Inheritance Vs Single Inheritance
- The ToString() Method
- Tips For Using Inheritance

Inheritance Concept

Inheritance is a parent/child relationship between two classes, in which the parent class is called the **Base Class**, while the child class is called the **Derived Class**. When defining a class, it inherits implicitly the class **Object**, which is the base class for all the **classes** in the .NET including C# language. For example, the source code:

```
class Shape {
    public Shape() {
            System.Console.WriteLine("It is the Shape’s constructor");
        }
    }
```

is equivalent to the following source code:

```
class Shape  : Object  {

    public Shape() {

            System.Console.WriteLine("It is the Shape’s constructor");

        }

    }
```

where the character " **:** " denotes the inheritance relationship.

Inheritance Concept

```
class Shape {
  protected int dimension;
  public Shape () {
    dimension = 1;
    Console.WriteLine("The Shape's Constructor");
        }
  public void SetDimention( int d ) {dimension =d; }
  ~Shape() {
     Console.WriteLine("Executing Destructor of class Shape");
    }
 }// end class Shape
```

When writing a new class, instead of writing all of its members, some members can be inherited from another class

```
 // derived class (inherits from Shape, extends Shape)
 class Circle : Shape  {
    int radius;
    public Circle () {
       radius=1;
       Console.WriteLine("The Circle's Constructor");
          }
    public void SetRadius(int r) {   radius =r;  }
    public void WriteDimesion() {
    Console.WriteLine("The Circle's dimension is " + dimension);  }
    ~Circle() {
    Console.WriteLine("Executing Destructor of class Circle");
     }
  }// end class Circle
```

The class **Circle** inherits the field **dimension** and the method **SetDimention** from the class **Shape**. It has also, the class **radius**, the method **SetRadius**, and the method **WriteDimesion**.

Inheritance Concept

```
/ class Circle without using inheritance
 class Circle  {
   int dimension;
   int radius;
 // contractor
   public Circle ( ) {
    radius=1;
    System.Console.WriteLine("The Circle's Constructor");
       }// end contractor
    public void SetDimention( int d ) {dimension =d; }
   public void SetRadius(int r) {   radius =r;  }
   public void WriteDimesion( ) {
   System.Console.WriteLine("The Circle's dimension is " + dimension);
         }// end WriteDimesion
    // destructor
   ~Circle(  )  {
      Console.WriteLine("Executing Destructor of class Circle");
      }// end destructor
}// end class Circle
```

Without using the inheritance, the developer must copy all the members of the class **Shape**, except the constructors and destructor, and paste them into the class **Circle**.

Inheritance Concept

- Having two or more copies of the same source code makes the maintenance of the source code difficult and risky.
  - It is possible for a developer to modify one copy of a method and forget to modify the other copy found in another class. This causes the same method to have two different behaviors by mistake.
- Using inheritance reduces the size of the source code, and also reduces the possibility of having two or more copies of the source code by mistake.

"protected" Access Specifier

- The access specifier **private** is used to protect a class member from being accessed by the members of other classes.

- The access selector **public** indicates that a class member can be used by members of the class to which the member belongs and by the members of other classes that have access to its container class.

- Sometimes we need to protect a class member from being used by other classes except the derived classes. C# (like some other Object Oriented programming languages) provides the access specifier **protected** that allows a class member to be used by the members of the class to which the member belongs and by the members of the derived classes.

"protected" Access Specifier

```
3. class Shape {
4.    protected int dimension;
```

- In the previous example, the **Shape** class includes the **dimension** field that its access specifier is **protected**. If this word is omitted, the access specifier becomes the default access specifier (i.e. private). This means that the Circle members cannot access this field.

- When we delete the **protected** word and try to execute the program, we will get the following error message:

```
'Shape.dimension' is inaccessible due to its protection level
```

- The message states that the **dimension** field is not accessible because of its protection level. The location of the error is indicated in the following line:

```
System.Console.WriteLine("The Circle's dimension is " + dimension);  }
```

- For other class members that have a **public** access specifier, we don't need to change them because Child class can access them like any other class.

Constructor and Destructor in The Inheritance

```
class Tester {
   public static void Main( ) {
  // creating cr object, an instance from Circle
  Circle cr = new Circle();
  //Calling SetDimention() method
  cr.SetDimention(2);
  //Calling WriteDimesion() method
   cr.WriteDimesion();
   //Releasing Object cr
   cr = null;
   GC.Collect();
   }
```

- When using inheritance, constructors and destructors cannot be inherited. It means that each one of the base class and the derived class has its own constructors and destructor.
- In our example, the **Circle** class does not inherit the contractor of the class **Shape**. The **Tester** class contains the method **Main** in which the object **cr** is created from the **Circle** class, and the **WriteDimesion** method is called.
- During execution, the default constructor of the **Shape** class and the constructor of the derived class **Circle** , both, will be called in order to create the **cr** object. Since the object **cr** is an object of the class **Circle** as well as an object of the class **Shape** at the same time, it can be used to call the **SetDimention()** method as follows:

```
cr.SetDimention(2);
```

Constructor and Destructor in The Inheritance

```
class Tester {
   public static void Main( ) {
  // creating cr object, an instance from Circle
  Circle cr = new Circle();
  //Calling SetDimention() method
  cr.SetDimention(2);
  //Calling WriteDimesion() method
   cr.WriteDimesion();
   //Releasing Object cr
   cr = null;
   GC.Collect();
   }
```

```
The Shape's Constructor
The Circle's Constructor
The Circle's dimension is 2
Executing Destructor of class Circle
Executing Destructor of class Shape
Press any key to continue . . .
```

- Then the method **WriteDimesion** is called to print the new value of the field **dimension**. The object **cr** is released by assigning it the **null** value and the **Collect** method from the **GC** (Garbage Collector) is called. After executing the program, the adjacent output appears.
- The destructors are not inherited. They are called in inverse order of the constructors. When the object **cr** is released, the **~Circle** destructor of the derived class **Circle** is called first, and then the **~Shape** of the base class **Shape**.

Inheritance and Private Constructor

- If a class has only one private constructor, no other class can inherit it.
- The private constructor is used where we do not want to create objects from the class outside the class. We explain how to use the private constructor in the following example:

```
class NoInstance {
    // Private Constructor:
    private NoInstance() { }

    public static double pi = Math.PI;  //3.1415...
}//end class NoInstance
```

- Adding a private constructor without any instruction inside it prevents the use of the default constructor which is automatically generated when the class is not defined.
- A private constructor can be used when the class contains only static members, and in such cases it is better to declare the whole class static.

Virtual Methods And Overriding

- A **Virtual Method** is one of the methods of the base class that is allowed to be overriding (changed its behavior) by another method in the derived class and both methods should have the same signature.

- The keyword **virtual** in a method signature indicates that the method is virtual.

- **virtual** is not used with any of the following words: **abstract, private, override, and static.**

Virtual Methods And Overriding

The class **TableTop** is derived from the class **Rectangle**. In addition to the inherited elements, it contains the **priceOneMeter** field for the price of a 1 m2, the **GetCost** method that returns the cost of the **TableTop**, and the **Display** method that overrides the default method in the base class and this method shows information about the current object;

```
class Rectangle {
protected double length, width;
public Rectangle(double l, double w) { length = l;  width = w;    }
public double GetArea() {  return length * width; }
public virtual void Display() {                    // virtual method
    Console.WriteLine("Length: {0}, Width: {1}, Area: {2}", length, width, GetArea()); }
}//end class Rectangle
class Tabletop : Rectangle {
private double priceOneMeter = 1;
    public Tabletop(double l, double w, double prc) : base(l, w) { priceOneMeter = prc;   }
    public double GetCost( ) {return (GetArea() * priceOneMeter);  }
    public override void Display() {                //overridden method
    //use base display and then show more data
    base.Display();
    Console.WriteLine("Cost: {0}", GetCost());
    }
}// end class Tabletop
```

We notice that the constructor of the derived class **TableTop** calls the constructor of the base class using the "**base**" keyword.

Virtual Methods And Overriding

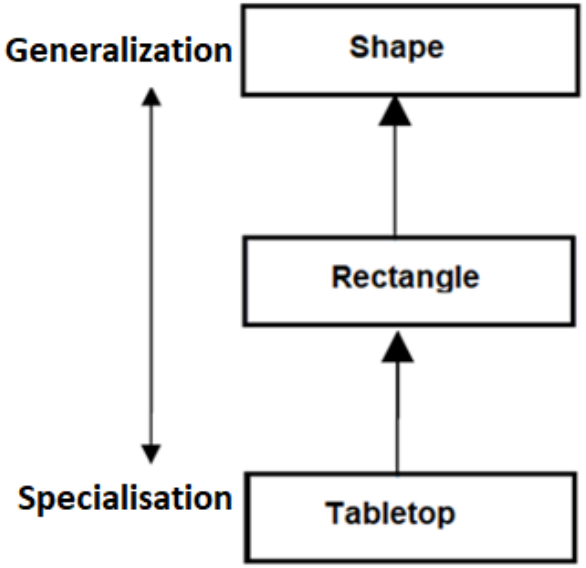
The overridden method **Display** in the derived class calls the virtual method **Display** from the base class using the "**base"** keyword.

```
class Rectangle {
protected double length, width;
public Rectangle(double l, double w) { length = l;  width = w;    }
public double GetArea() {  return length * width; }
public virtual void Display() {                    // virtual method
    Console.WriteLine("Length: {0}, Width: {1}, Area: {2}", length, width, GetArea()); }
}//end class Rectangle
class Tabletop : Rectangle {
private double priceOneMeter = 1;
    public Tabletop(double l, double w, double prc) : base(l, w) { priceOneMeter = prc;   }
    public double GetCost( ) {return (GetArea() * priceOneMeter);  }
    public override void Display() {                //overridden method
    //use base display and then show more data
    base.Display();
    Console.WriteLine("Cost: {0}", GetCost());
    }
}// end class Tabletop
```

Virtual Methods And Overriding

Notes

- The overriding method in the derived class and the overridden method in the base class must have the same signature, i.e., the same input arguments in terms of number and data types, and the same name;
- Overriding does not change the access specifier of the method being overridden;
- **Virtual, abstract, and override methods can be overridden**;
- static methods cannot be overridden;
- It is not possible to override private methods (for which the access is specified is private);
- Properties and indexers can be overridden.



Inheritance Hierarchy

The derived class can be a base class for other classes, the class **Rectangle** It is a base class for the **Tabletop** class and, at the same time, it is a derived class **Shape**.

- We call the **Rectangle** class the Direct Base Class of the **Tabletop** class, we call the **Shape** class the Indirect Base Class.
- In C#, a base class can have any number of derived classes, but **this language does not allow Multiple Inheritance**. That is, a derived class cannot have more than one direct base class, and this is what we call **Single Inheritance**.
- When designing inheritance between classes, the base class is usually more general than the directly derived class, which in turn is more general than the directly derived class, and so on. That is, the deeper we move in the inheritance hierarchy towards the derived classes, the more the specialization increases, and the generalization increases in the opposite direction.

Inheritance Hierarchy

- The derived class and base class are related with "**is-a**" relationship, which means that any object of the derived class is an object of the base class.
- In the source code shown below, the class **Teacher** inherits the class **Person**. We create **myTeacher** object from the class **Teacher** and the object **p1** from the class **Person**. We can assign the object **myTeacher** to the object **p1** because every object from the **Teacher** class is also an object from the **Person** class. For the same reason, we can create the object **p2** from the **Person** class using the constructor of the **Teacher** class.

```
Teacher myTeacher = new Teacher(23, "OOP", "Saly");
Person p1 = new Person();
p1 = myTeacher;
Person p2 = new Teacher(25, "Math", "Khaled");
```

Inheritance Hierarchy

- Now, let's assign the object **p2** to the **myTeacher** object, we get the following error message because it is not possible to implicitly convert between the **Person** data type and **Teacher** data type:

Cannot implicitly convert type 'Inheritance.Person' to 'Inheritance.Teacher'. An explicit conversion exists (are you missing a cast?)

```
Teacher myTeacher = new Teacher(23, "OOP", "Saly");
Person p1 = new Person();
p1 = myTeacher;
Person p2 = new Teacher(25, "Math", "Khaled");
myTeacher = p2;
```

- In order to correct the previous error, an explicit type conversion (type casting) must be done between the **Person** data type and the **Teacher** data type.

```
myTeacher = (Teacher) p2;// accepted
```

Overriding the ToString Method

- The **ToString** method exists for all objects; it is used to return a string containing information about the object that invokes it.
- If its behavior is not modified, the **ToString** method returns the data type of the object that invokes it.
- To show how to use it, in the following instructions, we override the **ToString** method to return information as a string such as name, age, and subject. It can be called within another method using the keyword "**this**".

```
public override string ToString() {
return string.Format("Hello! I am a {0} ,  my age is {1}, Te
acher of {2}.", Name, age, subject);
}

public void PrintInfo(){
Console.WriteLine(this.ToString());         }
```

Tips For Using Inheritance

- Despite many benefits of inheritance, such as the reusability of the source code and decreasing its size, it imposes some duties on programmers:
  - Base class and child classes are tightly coupled. Hence, If we change the code of parent class, it will get affects to the all the child classes
    - When an error occurs in the base class, the error will be inherited to the derived classes;
  - In the class hierarchy, many data members remain unused and the memory allocated to them is not utilized.
- Programmers can prevent the class from being inherited by declaring it as **sealed**.
- The concepts of method overloading and method overriding should not be confused. **Overloading** is the ability to have multiple methods <u>within the same class</u> with the same name, but with different parameters. **Overriding**, on the other hand, is the ability to redefine the implementation of a method in a class that inherits from a parent class. Each concept of them represents a type of **Polymorphism**.

Summary

- Base Class Vs Derived Class;
- "protected" Access Specifier;
- Constructor and Destructor in The Inheritance;
- Using "base“ Keyword;
- Overriding the ToString Method;
- Multiple Inheritance Vs Single Inheritance.

Thank you very much for attending

You can kindly ask questions